GATE DA · Database Management and Warehousing

Exam notes · Version 1 · 11 October 2026

Eight chapters. Forty reusable problem recipes. Original worked examples and explanations for 23 DA and 21 selected CS previous-year questions. The companion practice book has 80 original problems, including four 20-mark mixed tests. Answers are in a separate solutions guide.

Start with an ordinary table, a row, a reference or a box of records. Use it to understand the rule. Then solve a small example, a changed-condition exercise and finally a mixed question without a method label.

The target is reliable exam reasoning. There is no backend project or long lecture sequence in this pack. Twenty marks is a preparation assumption, not an official promise about the section's weight.

Read this first

The topic checklist comes from Section 5 of the official 2027 DA syllabus. Read the original PYQ in its linked official paper before using the explanation. Page references count from the PDF's first page, which can differ from the printed footer.

Original examples and problems are labelled W, P or T. They are not official exam questions. The 44 worked PYQs cover every DA question in our 2024–2026 database/warehouse/preprocessing index plus 21 selected CS questions from six papers. This is not an exhaustive audit of all historical CS/IT questions or a guarantee of 20/20.

The chapters were generated, checked and locked in order. Numerical and finite-structure exercise checks use question inputs without reading the generated solution files. I reviewed the explanations, source keys and PDF pages. This is author review plus computational verification, not independent editorial certification. Source/key discrepancies are stated where they occur.

Contents and study route

Chapter	What you need to do	90-minute sessions
1 · Relations, keys, constraints	identify permitted rows and safe references	2
2 · Algebra and calculus	track sets, matches and witnesses	3
3 · SQL	trace rows, groups, NULL and nested queries	4
4 · Dependencies and normal forms	close attributes, find keys, test decomposition	5
5 · ER models	translate a story into tables and constraints	1
6 · Files and indexes	calculate capacities, repair trees, count I/O	3
7 · Warehouses	identify grain, levels and sufficient summaries	1
8 · Preprocessing	scale, bin, sample and count encoded size	1

This suggested route is 30 hours of chapter work. Allow four further hours for the mixed tests and their corrections. Reduce reading time for topics you can already explain and solve. Give more time to errors, not to lecture completion.

Notes pages: Chapter 1: 5; Chapter 2: 8; Chapter 3: 13; Chapter 4: 18; Chapter 5: 24; Chapter 6: 27; Chapter 7: 32; Chapter 8: 35. Use PDF bookmarks for direct navigation.

Within a session: use roughly 25 minutes for the motivating example and definitions, 20 for a worked PYQ, 30 for your own solutions and 15 for corrections. Read a hint only after writing your first attempt. After an error, record the missed condition: duplicate, NULL, strict inequality, candidate domain, missing FD, node layout or aggregation level. Reattempt a fresh variation later.

Conventions

- Classical relational algebra uses sets. Ordinary SQL SELECT keeps duplicates unless a clause removes them.
- SQL WHERE retains TRUE, not UNKNOWN. NULL is not zero or an ordinary equal value.
- Normal-form examples use atomic domains and exclude nontrivial empty-determinant FDs unless explicitly discussed. See the convention-sensitive CS-2022-Q14 explanation.
- A natural join matches every shared attribute name. A key is a schema rule, not a coincidence in today's rows.
- B+ capacities, split rules, routing and height conventions are stated separately. Use the question's conventions.
- Byte layouts ignore overhead only when the question does. A data fetch and identifying its block are different tasks.

Find the recipe

Read the requested output first. The recipe names below identify the operation, not a formula to apply without checking its conditions.

Recipe	Requested task	First check
R01	Identify keys and distinguish schema from instance	declared uniqueness, not instance coincidence
R02	Foreign key operations	child→parent and referential action
R03	Possible table sizes and duplicate identifiers	unique side and required matches
R04	Evaluate an algebra expression	attributes at every node
R05	Join cardinality and bounds	frequencies per shared value
R06	Translate some none and every	candidate domain and empty requirements
R07	Query equivalence and counterexamples	small counterexample and rewrite conditions
R08	Evaluate tuple relational calculus	output tuples versus witnesses
R09	Trace a query block	logical stages and duplicates
R10	NULL and aggregate questions	NULL inputs and empty aggregates
R11	GROUP BY and HAVING	full grouping tuple
R12	Scalar and correlated subqueries	outer-row dependence and ties
R13	EXISTS IN ANY and ALL	NULL and empty subquery
R14	Outer joins and self joins	ON before WHERE; independent aliases
R15	Attribute closure and implied dependencies	whole determinants available
R16	Find all candidate keys	compulsory attributes and minimality
R17	Count superkeys	overlap between key supersets
R18	Minimal cover and equivalence	closure after removing tested rule
R19	Highest normal form	all keys and all prime attributes
R20	Binary lossless decomposition	intersection closure
R21	Nonbinary losslessness	distinguished symbols in full tableau
R22	Dependency preservation	project F+, not only written F
R23	Construct a decomposition	lossless and preservation separately

Recipe	Requested task	First check
R24	Read a diagram and bound relationship sizes	both lower and upper partner bounds
R25	Map entities and relationships to tables	one row meaning, key and reference
R26	Minimum relations and missing information	which facts remain independent
R27	Blocking factor and storage size	usable bytes and unspanned packing
R28	Single and multilevel index size	what each entry represents
R29	B+ tree capacities	internal versus leaf layout
R30	Tree height and number of nodes	levels/edges and root exceptions
R31	Tree insertion and deletion	copy versus promote; repair occupancy
R32	Index choice and access cost	range/equality, cache and covering
R33	Schema and grain	fact grain and join multiplicity
R34	OLAP operations	before/after grouping or filter
R35	CUBE groups and hierarchy counts	sets, observed groups or level choices
R36	Measures and correct roll-up	sufficient summaries and dimension additivity
R37	Numeric normalization	scale, units and zero denominator
R38	Discretization and bin smoothing	intervals, ties and replacement rule
R39	Sampling	sampling unit, allocation and weights
R40	Compression	lossless/lossy, overhead and ratio direction

Chapter 1 · Relations, keys and constraints

A college keeps a register with student number, name and department. Two students can share a name. A student number identifies one student. A separate enrolment list records which courses each student takes.

Student number	Name	Department
11	Asha	Physics
12	Ravi	Mathematics
13	Asha	Chemistry

Vocabulary. A relation is a set of tuples, presented as rows. An attribute is a named column. Its domain is the set of permitted values. A schema specifies attributes and constraints; an instance is the current collection of rows. Degree counts attributes; cardinality counts tuples. SQL tables may contain duplicate rows unless constraints prevent them.

A superkey is an attribute set that uniquely identifies a tuple in every permitted instance. A candidate key is a minimal superkey. Minimal means that no proper subset is a superkey; it does not mean the shortest key. A primary key is a selected candidate key. A prime attribute belongs to at least one candidate key. A foreign key references a permitted unique identifier in another table; referencing it does not make the foreign key unique.

R01 Identify keys and distinguish schema from instance

Use when: a question asks which attributes are keys.

1. List the stated uniqueness rules and dependencies.
2. Check whether a proposed set determines all attributes. Use closure if dependencies are supplied.
3. Remove attributes to test minimality.
4. Separate candidate keys from their larger superkeys.

Seeing three different names in a table does not establish that Name is a schema key. Conversely, two identical names disprove uniqueness for that instance. In SQL, a primary key is unique and non-null. Nullable UNIQUE constraints require care: their treatment of NULL is dialect-dependent.

Worked example W01. In the college register above, StudentID is the declared identifier. {StudentID} and {StudentID, Name} both identify rows. Only the first is minimal. Name is not a key: students 11 and 13 share it. If the current three department names differ, that coincidence still does not make Department a schema key. A relation with three attributes and three rows has degree 3 and cardinality 3. Adding a fourth student changes the cardinality, not the degree.

Check before moving on. Can a candidate key contain two columns? Yes. Can one table have two different candidate keys? Yes. Primary chooses one of them.

R02 Foreign key operations

Use when: insertion, deletion or updating may violate a reference.

1. Draw an arrow from the child reference to the parent identifier.
2. For child insertion/update, check that each required non-null reference has a matching parent.

3. For parent deletion/key update, check whether children still reference the old value.
4. Apply stated CASCADE, SET NULL or other actions. Without such an action, do not silently repair a broken reference.
5. Check other constraints independently.

Adding a parent or deleting a child cannot create a dangling reference under that foreign key alone. The operation may still violate another constraint. A foreign key is not automatically NOT NULL.

Worked example W02. Let Department contain IDs {10,20}. Student has rows (11,10), (12,20), (13,NULL), and Student.DeptID is a nullable foreign key. Inserting (14,30) creates a missing reference and fails. Inserting Department 30 first makes that reference valid. Deleting Student 11 is safe for this foreign key. Deleting Department 20 leaves Student 12 without a parent and fails unless a specified referential action repairs it. Under SET NULL it changes Student 12's reference to NULL, provided that column permits NULL. Inserting a parent is safe for this reference but can still violate the parent's primary key.

Check before moving on. Name the child and parent before deciding whether an update is safe. Do not assume a foreign key is unique or non-null.

R03 Possible table sizes and duplicate identifiers

Use when: a question asks for minimum/maximum rows or join sizes under keys.

Translate each key into an upper bound on matching rows. A child can reference a parent many times unless uniqueness prevents it. Total participation supplies lower bounds; keys often supply upper bounds. Construct an instance achieving a proposed bound before calling it attainable.

Worked example W03. Four purchase rows reference CustomerID 1,1,2,NULL. Customer has IDs 1 and 2, with CustomerID a primary key. The inner join returns three rows: each non-null purchase has exactly one parent, while NULL has no equality match. The left join returns four rows, including the unmatched purchase. Joining in the other direction does not give one row per customer: customer 1 appears twice. If all four purchase references were non-null valid foreign keys, the inner join would have exactly four rows. Without the foreign key, the upper bound would still be four, but the lower bound could be zero.

Check before moving on. State which side is unique and whether every reference must match. Then count matches, not just source rows.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2025-Q56 · Candidate keys and NULL

Official question, PDF page 30. Published key: **B; C**.

Customer's primary key is (name,email). Its aadhaar column is only a foreign key, so repeated references are allowed: A is false. Phone has no NOT NULL restriction: B is true. Person.aadhaar is the primary key and therefore a candidate key: C is true. A primary key cannot contain NULL: D is false. Do not transfer a uniqueness constraint from the referenced table to the referencing column.

DA-2026-Q26 · Safe foreign-key operations

Official question, PDF page 18. Published key: **A; B**.

Inserting into the referenced parent cannot remove an existing target. Deleting from the referencing child cannot create a dangling reference. These are A and B. Inserting a child can reference a missing parent. Deleting a parent can leave an existing child without its target. Those operations are not always safe. The conclusion concerns the named foreign key alone, not every possible constraint.

CS-2024-set1-Q22 · First normal form and keys

Official question, PDF page 15. Published key: **A; C**.

Under the exam's atomic-value convention, 1NF permits a multi-attribute key, so A is true. It does not prohibit foreign keys, so B is false. A composite attribute must be represented through its atomic components rather than one structured cell, so C is the intended true statement. Several candidate keys are permitted, so D is false. Atomicity concerns a column's chosen domain; it is different from a key using several columns.

CS-2023-Q16 · Degree versus cardinality

Official question, PDF page 13. Published key: **A**.

Count the attributes in the schema. That count is the degree. Count tuples in an instance for its cardinality. The option identifying the attribute count is A. Neither the number of possible values nor the number of currently stored rows defines degree.

Before the exam

Be able to distinguish schema from instance, minimal from minimum, and candidate from primary key. For a reference operation, identify what could become missing. For a row-count bound, state the uniqueness and participation assumptions that make the bound attainable.

Chapter 2 · Relational algebra and tuple calculus

Suppose a college has Student(StudentID, Name), Enrolment(StudentID, CourseID) and Required(CourseID). We want the students enrolled in every required course. First solve it by checking each student's checklist. The symbols should describe that same check.

Vocabulary. Selection keeps rows satisfying a predicate. Projection keeps specified columns and removes duplicate tuples in classical algebra. Product forms all pairs. A theta join keeps pairs satisfying its condition. A natural join matches on every shared attribute name. Rename makes roles and attribute names explicit. Union, intersection and difference require compatible schemas. Division expresses an every-member requirement.

R04 Evaluate an algebra expression

1. Parse the parentheses and draw the expression tree.
2. Write the output attributes at every node.
3. Evaluate the innermost operations and show intermediate rows.
4. Remove duplicates after projection and other set operations.
5. Apply the final projection and count the resulting tuples.

For a natural join on (P,Q), matching P alone is insufficient. Same-named columns can make a join more restrictive than the story suggests.

Worked example W04. Take R(Student,Course)={(Asha,Math),(Asha,Physics),(Ravi,Math)} and S(Course,Room)={(Math,10),(Physics,20)}. R natural-join S matches Course. Its rows are (Asha,Math,10), (Asha,Physics,20), (Ravi,Math,10). Selection Room=10 leaves the first and third rows. Projection Student returns {Asha,Ravi}. Projection Course instead returns just {Math}: the two identical projected values become one tuple. To see the other operators, rename two copies of Student as S1 and S2 before taking their product; a two-row relation produces four ordered pairs. Union combines compatible tuple sets, intersection keeps common tuples, and difference keeps tuples in the first set but not the second.

Check before moving on. Write the attributes at every expression-tree node. A natural join uses every shared attribute name, and projection uses sets.

R05 Join cardinality and bounds

For an equality join on J, count rows per join value: the result size is the sum of nR(j) times nS(j). If J is a key on one side, each row on the other side has at most one match. A valid non-null foreign key to that key guarantees exactly one match per child. A nullable reference does not. For products, multiply input sizes. For set projections, count distinct projected tuples.

Worked example W05. R has join-value counts a:2 and b:1. S has counts a:3, b:2 and c:1. There are 2×3+1×2=8 matching pairs. The unmatched c contributes zero. If S's join column becomes unique, each R row has at most one match, so the join has at most 3 rows. A non-null foreign key from R to S makes that upper bound exact. The frequency calculation and the key-based bound answer different versions of the same question.

Check before moving on. Do duplicates describe distinct tuples with the same join value? Keep every matching pair before the final set projection.

R06 Translate some none and every

- Some: at least one matching row satisfies the condition.
- None: no matching row satisfies the condition.
- Every: no relevant row fails the condition.

For R(StudentID,CourseID) divided by Required(CourseID):

1. Take candidate students from the projection of R.
2. For each candidate, form the set of courses associated with it.
3. Keep it if Required is a subset of that set.

Empty Required makes the condition true for every candidate from the specified candidate domain. It does not invent students missing from that domain. SQL implementations using NOT EXISTS need explicit NULL reasoning.

Worked example W06. Enrolment={(1,M),(1,P),(2,M)} and Required={M,P}. Student 1 covers both required values; student 2 misses P. Division returns {1}. Using candidate set C={1,2}, first form C×Required: (1,M),(1,P),(2,M),(2,P). Subtract Enrolment to get the missing pair (2,P). Project its student to obtain {2}; C minus {2} is {1}. If Required is empty, there are no missing pairs and division returns {1,2}. If candidates instead come from Student={1,2,3}, a NOT EXISTS query returns all three. The candidate domain must be stated.

Check before moving on. Rewrite every as no required value is missing. Test an empty requirement and an absent candidate.

R07 Query equivalence and counterexamples

Use when: two expressions are claimed to return the same result.

Track schemas, matching columns and quantifiers. Try a tiny input containing duplicates, an unmatched row, two matches or an empty relation. One differing output disproves equivalence. Matching on a few examples does not prove equivalence. To justify a valid rewrite, use an operator law with its conditions; for example, pushing a selection into an input requires that input to contain all referenced attributes.

Worked example W07. Consider the claim that Team is contained in Defender intersection Forward whenever it is contained in Defender union Forward. Let Team={Asha}, Defender={Asha}, Forward be empty. Asha belongs to the union but not the intersection. One row disproves the claim. A valid pushdown example is selection Room=10 on R(Student,Course) joined with S(Course,Room): the selection can move into S because S supplies Room. Moving it into R is not well-formed because R has no Room attribute. A counterexample disproves a claimed law; several matching examples do not prove it.

Check before moving on. Try one missing match or one extra tuple. For a claimed valid rewrite, state the operator law and its conditions.

R08 Evaluate tuple relational calculus

A predicate is a condition that can be true or false. A tuple variable stands for a row. An existential quantifier means at least one witness; a universal quantifier means every row in its stated domain.

1. Identify the output tuple and its domain.
2. Give each quantified variable a separate role.
3. For each candidate output tuple, test the conditions and search for witnesses.

4. Count qualifying output tuples, not witness combinations.

Translate a universal condition using “no counterexample exists.” Do not assume two variables are distinct unless required. An existential condition may be redundant because the candidate tuple itself supplies a witness. Safe calculus queries depend on the database's active domain and fixed query constants; finite output alone is not an adequate general definition of safety.

Worked example W08. Let X(P,Q)={(1,a),(2,b),(3,c)} and Y(P,S)={(1,10),(1,20),(2,30),(3,1)}. The predicate for an output row x is: x belongs to X and some y in Y has y.P=x.P and y.S>1. x=(1,a) has two witnesses, x=(2,b) has one, and x=(3,c) has none. There are 2 output tuples, not 3 witnesses. Adding a condition that some z in X has z.P=x.P changes nothing: z=x already works. In notation, $\{x \mid x \in X \wedge \exists y \in Y\ (y.P = x.P \wedge y.S > 1)\}$ states precisely that row test. Unrestricted $\{x \mid x \notin X\}$ is not safe over an unbounded tuple universe.

Check before moving on. Separate output tuples from witnesses. Do not assume two variable names force two different rows.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q26 · Containment and union

Official question, PDF page 16. Published key: **C**.

Let T,D,F be the projected name sets. Every Team name must appear in at least one of D,F, so T must be a subset of D union F. Test this by checking whether T minus (D union F) is empty. That is C. A demands both roles, which is too strong. B and D reverse the inclusion: they can reject an extra player even when all Team names are covered.

DA-2025-Q17 · Read joins as a sentence

Official question, PDF page 12. Published key: **C**.

Filter makers to ABC, join cars on model, and filter those cars to red. Join ownership on serial. The final projection contains owners of red ABC cars: C. There is no count or requirement that an owner have several cars. Both filters are required, not an OR. The output is owner identifiers, not car rows.

DA-2025-Q62 · Division

Official question, PDF page 32. Published key: **1 to 1**.

List the four required branches, then make one branch set for each account holder from the displayed ownership tuples. Only Karteek covers all four. Extra accounts at a covered branch do not replace a missing branch. The divided relation therefore contains one owner. Use the original displayed tables to fill the four-column checklist before counting.

DA-2026-Q42 · Nested natural joins

Official question, PDF page 27. Published key: **D**.

X natural-join Y matches both P and Q. After the left selection, the surviving tuple is (P3,Q3,R2,S1). On the right, Y natural-join Z followed by S>1 leaves tuples with P1 and S2 or S5. The outer natural join shares P,Q,S. The left P3 tuple has no right partner. Its result is empty, so projection also has zero rows: D. Matching only P in the first join would already be wrong.

DA-2026-Q59 · Witnesses and an official range

Official question, PDF page 38. Published key: **2 to 3**.

For the printed condition S>1, the X rows with P1 and P2 qualify. P1 has two Y witnesses but still contributes one output tuple. P3's Y value is 1, so it fails the strict inequality. The additional X witness can be the output tuple itself. This derives 2. The published accepted range is 2 to 3; retain that range when grading the historical exam. This guide does not change >1 into ≥1 to manufacture the other endpoint.

CS-2024-set1-Q35 · Join then filter

Official question, PDF page 21. Published key: **2 to 2**.

Join on A. The matched triples are (10,20,90), (30,40,45), and (30,50,45). Apply B<C: the first two pass and the third fails. A=20 and A=50 have no S partner; A=40 has no R partner. The final count is 2.

CS-2024-set2-Q45 · Three distinct witnesses

Official question, PDF page 31. Published key: **B**.

Use three renamed copies of the person relation. Three copies require two binary Cartesian-product operations. Require all three pairwise identifier inequalities as well as the shared target condition. Without inequalities, one person could occupy all three roles. The option with two cross products is B.

CS-2025-set1-Q39 · Bind calculus variables

Official question, PDF page 39. Published key: **A**.

The output pname must come from a player p. A team t must exist with p.tid=t.tid and t.tname='MI'. A binds both variables to the right relations and includes the link. B and D give variables attributes their relations do not have. C omits the identifier match and would admit every player whenever any MI team exists.

CS-2022-Q25 · Division and missing pairs

Official question, PDF page 20. Published key: **A; B**.

A divides the ownership pairs by all Brand identifiers. B removes from the ownership candidate set anyone with a missing candidate×Brand pair. Both express the same requirement. C only asks for brands already appearing in Own and can omit an unowned Brand. D builds a product that ignores each employee's actual ownership. The candidate domain is employees occurring in Own. To include employees with no ownership when Brand is empty, a different expression must explicitly use Employee as its candidate domain.

Before the exam

Translate the requested output into rows, sets or witnesses before counting. Keep intermediate attributes visible. Test every/none expressions on an empty requirement. For equivalence claims, a small counterexample is often faster than a large calculation.

Chapter 3 · SQL: rows, groups and nested questions

A shop records purchases. Asha has two purchases, Ravi one. Joining customers to purchases gives Asha two output rows. The join has produced purchase matches, not one row per customer. This distinction explains many count questions.

Vocabulary. A query block is one SELECT statement with its clauses. An alias names a table's role. A group collects rows with the same grouping values. An aggregate summarizes those rows. A correlated subquery refers to an outer query's row.

R09 Trace a query block

Use logical evaluation order as a reasoning aid: FROM/JOIN and ON, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, then the stated row limit. The engine may use another execution plan.

1. Produce matching rows, preserving SQL duplicates.
2. Keep only rows where WHERE is TRUE.
3. Form groups and compute aggregates.
4. Apply HAVING, then calculate selected expressions.
5. Remove duplicates only if DISTINCT or a set operator requires it.

UNION normally removes duplicates; UNION ALL preserves them. Do not apply relational-algebra projection semantics to ordinary SQL SELECT.

Worked example W09. Customer contains (1,Asha),(2,Ravi). Purchase contains (1,10),(1,20),(2,30). Joining on the ID produces Asha twice and Ravi once. WHERE amount>15 leaves (Asha,20),(Ravi,30). SELECT Name returns two rows. Without that WHERE, SELECT Name returns Asha,Asha,Ravi; SELECT DISTINCT Name returns Asha,Ravi. Trace a query block in stages even if the database executes it differently. Classical algebra and ordinary SQL disagree about duplicate removal here.

Check before moving on. Identify duplicate-removal clauses explicitly. SELECT alone does not remove duplicates.

R10 NULL and aggregate questions

NULL indicates missing/unknown information. A comparison with NULL ordinarily produces UNKNOWN, including NULL = NULL. Use IS NULL to test it. WHERE keeps TRUE, excluding FALSE and UNKNOWN.

Write the three-valued truth table when needed. FALSE AND UNKNOWN is FALSE; TRUE OR UNKNOWN is TRUE. COUNT(*) counts rows. COUNT(column) counts non-null values. COUNT(DISTINCT column) counts distinct non-null values. SUM and AVG ignore NULL inputs. On empty input, COUNT returns zero and ordinary SUM/AVG return NULL. An ungrouped aggregate still produces one row; GROUP BY on empty input produces no groups.

Worked example W10. For Value={10,NULL,10,20}, COUNT(*)=4, COUNT(Value)=3, COUNT(DISTINCT Value)=2, SUM(Value)=40 and AVG(Value)=40/3. The average divides by three observed numbers, not four rows. On no matching rows, an ungrouped aggregate returns one row containing COUNT=0 and SUM=NULL. A grouped aggregate returns no groups. NULL=10 and NULL=NULL are UNKNOWN, not FALSE. WHERE accepts only TRUE. FALSE AND UNKNOWN is FALSE; TRUE AND UNKNOWN is

UNKNOWN; TRUE OR UNKNOWN is TRUE; FALSE OR UNKNOWN is UNKNOWN; NOT UNKNOWN stays UNKNOWN.

Check before moving on. Write the denominator of AVG and distinguish an empty group from an absent group.

R11 GROUP BY and HAVING

Partition the surviving input rows by the full grouping tuple. Write each group's members, then its aggregates. Apply HAVING to the group. Keep the requested output columns. In standard exam SQL, selected nonaggregate columns must be valid grouping columns or otherwise permitted by the question's stated semantics. Do not accept a permissive engine's arbitrary value as an exam rule.

Worked example W11. Sales rows are (A,10),(A,20),(B,7),(B,NULL). WHERE amount>8 leaves only A's two rows. GROUP BY team creates one A group; SUM=30 and COUNT(*)=2. HAVING SUM(amount)>25 keeps that group. Moving the condition into HAVING does not mean filtering individual amounts. If the query instead groups all four rows, B's sum is 7 and its COUNT(amount) is 1, while COUNT(*) is 2. Always write a group's members before its aggregate.

Check before moving on. WHERE tests rows; HAVING tests groups. Write both stages separately.

R12 Scalar and correlated subqueries

First ask whether the inner query depends on the outer row. If not, evaluate it once. If it does, freeze one outer row, substitute its values, evaluate the inner query and test the outer condition. Repeat for each outer row. A scalar subquery expects at most one row; multiple rows are not silently averaged. An aggregate without GROUP BY returns one row, although its value can be NULL.

For ranking by comparisons, count rows strictly above and strictly below each value. Tied rows can affect counts even if they do not satisfy strict inequalities. For balances 5,5,10,15,18, the condition “number above >= number below” retains the first three rows. Do not replace it with a rank shortcut without checking ties.

Worked example W12. For balances 5,5,10,15,18, count values strictly above and below each row. A balance-5 row has 3 above and 0 below; balance 10 has 2 above and 2 below; 15 has 1 above and 3 below; 18 has 0 above and 4 below. The test above≥below retains the two 5 rows and the 10 row: 3 output rows. For an uncorrelated example, SELECT MAX(amount) from one branch gives a single threshold; compare every outer amount with that one value. An aggregate on an empty branch gives NULL, so amount>MAX(empty) keeps no rows.

Check before moving on. Freeze one outer row before evaluating a correlated subquery. Count ties using the exact inequality.

R13 EXISTS IN ANY and ALL

EXISTS tests whether the inner query returns any row. It does not test whether the selected value is non-null. NOT EXISTS tests absence. IN is an equality-membership test and can be UNKNOWN when NULL is involved. NOT IN can therefore fail to retain an apparently unmatched row.

For ANY, test the comparison against each value: any TRUE gives TRUE; otherwise UNKNOWN can remain. For ALL, any FALSE gives FALSE; otherwise UNKNOWN can remain. Over empty input, ANY is FALSE and ALL is TRUE. EXISTS over empty input is FALSE.

Worked example W13. Applicant IDs are 1,2,3. Rejected IDs are 2,NULL. For ID 1, NOT IN tests NOT(1=2 OR 1=NULL): NOT(FALSE OR UNKNOWN) is UNKNOWN. It is filtered out. ID 2 is explicitly rejected, and ID

3 is also UNKNOWN. NOT IN returns no applicants. A correlated NOT EXISTS asks whether a rejected row actually matches the applicant ID. Only 2 has such a row, so it returns 1 and 3. On an empty subquery, EXISTS and ANY are FALSE, while ALL and NOT EXISTS are TRUE. For 5>ALL{2,NULL}, no comparison is FALSE but one is UNKNOWN, so the result is UNKNOWN.

Check before moving on. Check NULL and empty input before replacing NOT IN with NOT EXISTS or ALL with MAX.

R14 Outer joins and self joins

For an outer join, make ON matches first, then add preserved unmatched rows with NULL on the missing side. Apply WHERE afterwards. A WHERE test on the missing side can remove these rows.

For a self join, draw two independent copies of the table and label their aliases. Form pairs satisfying the predicate. The two aliases can refer to the same underlying row unless an inequality excludes that case. Compute per-row matches before grouping. Check whether a predicate intended to match a team instead matches only the same employee.

Worked example W14. Asha has purchases 10 and 20; Ravi has none. LEFT JOIN Purchase ON matching_ID AND amount>15 yields (Asha,20),(Ravi,NULL). Put amount>15 in WHERE instead: the unmatched Ravi row is now rejected because NULL>15 is UNKNOWN. For a self join on team, aliases A and B represent independent employee roles. An employee in a three-person team has three B matches, including itself. Adding A.EmpID=B.EmpID reduces it to one. Grouping by A.EmpID then counts 1 rather than the team size.

Check before moving on. An unmatched outer-join row is added after ON but before WHERE. Self joins need separate role labels.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q31 · Filter a join

Official question, PDF page 19. Published key: **3 to 3**.

The Jaipur references are Raider IDs 2,1,6. Their raid points are 219,250,215, all strictly above 200. Each reference matches one Raider, so three joined rows survive. Hyderabad's qualifying raider is irrelevant because the team filter also has to hold.

DA-2025-Q33 · One scalar threshold

Official question, PDF page 18. Published key: **3 to 3**.

Evaluate the inner branch maximum first: 65,000. The outer values above it are 90,000,80,000,70,000. Exactly three rows qualify. The 65,000 row fails the strict **>** test. The subquery supplies one value, not a relation to join with every account.

DA-2026-Q51 · Team size per employee

Official question, PDF page 34. Published key: **A**.

Group by TeamID: teams 8,7,9 have sizes 3,2,1. A joins these summaries back to each employee, giving sizes 3,3,3,2,2,1. B additionally requires identical EmpID, so each employee only matches itself and gets 1. C groups by the primary key and also gets 1. D's derived table does not expose TeamID, so B.TeamID cannot be referenced. Only A works.

DA-2026-Q60 · Correlated counts and ties

Official question, PDF page 38. Published key: **3 to 3**.

For balances 5000,5000,10000,15000,18000, the strict above/below counts are (3,0),(3,0),(2,2),(1,3),(0,4). Apply the printed count comparison to each outer row. The first three pass, giving 3. The tied 5000 rows remain two separate account rows even though neither is below the other.

CS-2025-set1-Q55 · Membership then minimum

Official question, PDF page 55. Published key: **26 to 26**.

Mark has coach ID 102, which selects team 10. Members of team 10 are player IDs 1 and 3. Their ages are 31 and 26. MIN(age)=26. Player 2's age 24 is smaller but belongs to team 30, so it is not eligible. Follow identifier links before taking the minimum.

CS-2025-set2-Q54 · Existence formulations

Official question, PDF page 42. Published key: **A; C; D**.

A tests for at least one matching course enrolment. C expresses the same condition with COUNT(*)>0. B uses SIZEOF, which is not the SQL construct in the question. The official key also accepts natural join D, which joins on rollno. Important bag-semantics qualification: if repeated (rollno,courseno) enrolment rows are allowed, D can repeat a student's name while A and C emit it once per Student row. Thus the official intended enrolment interpretation is not an unconditional bag-equivalence theorem. Use DISTINCT or a

suitable enrolment key when that distinction matters.

CS-2022-Q56 · No missing required course

Official question, PDF page 38. Published key: **2 to 2**.

Department D01 requires courses C11 and C12. Subtract each student's registered courses from {C11,C12}. S01 and S04 leave an empty difference; S02 and S05 miss C12; S03 misses both. NOT EXISTS retains S01,S04, giving 2. The paper prints NOT EXIST; read the intended NOT EXISTS construct. Course membership, not the student's own department, determines this predicate.

CS-2023-Q61 · Two simultaneous row filters

Official question, PDF page 64. Published key: **2 to 2**.

Apply gender='F' and marks>65 to each displayed row. The two Aliya rows with marks 70 and 80 pass. Swati's 65 fails the strict inequality. COUNT(*) counts the two rows, not one distinct name. Result: 2.

Before the exam

Before counting, mark bag versus set semantics, NULL, the grouping tuple and each strict inequality. For nested queries, determine whether the inner result is a scalar, a set or an existence test. For an MSQ, test every option independently, including whether it is valid SQL.

Chapter 4 · Dependencies, keys and normal forms

A library ledger records BookID, Title, PublisherID and PublisherAddress. BookID determines the title and publisher. PublisherID determines the publisher's address. Repeating the address for every book makes a correction easy to miss. We will use these rules to decide what identifies a row and whether splitting the ledger preserves its information.

Vocabulary. X determines Y, written X -> Y, means that any two permitted tuples agreeing on X also agree on Y. X and Y can each be sets of attributes. A functional dependency is a rule about permitted instances. It cannot be established merely because a small displayed table happens to satisfy it. A trivial dependency has Y contained in X. X+ is the attribute closure of X under the given dependencies. F+ is the set of dependencies implied by F.

R15 Attribute closure and implied dependencies

Start X+ with X. Whenever a dependency's entire left side is inside the current closure, add its right side. Repeat until nothing changes. X -> Y follows exactly when Y is contained in the final closure.

Example: A -> B, B -> C on ABCD gives A+ = ABC. D is still missing. AD+ = ABCD.

Worked example W15. For R(ABCD), F={A→B,B→C}, start A+={A}. A→B adds B; B→C adds C. No rule adds D, so A+={A,B,C}. AD+ starts {A,D}, then reaches all four attributes. Thus A→C follows, A→D does not, and AD is a superkey. A dependency's whole left side must already be present: knowing A alone does not activate AB→D. Armstrong's basic rules are reflexivity (a set determines its subsets), augmentation (add the same attributes to both sides), and transitivity (chain two determining rules). Closure repeatedly applies those consequences.

Check before moving on. Is every left-side attribute available before a rule fires? Stop only at a fixed point.

R16 Find all candidate keys

1. Put every attribute absent from all right sides into the compulsory set: none can be obtained from another attribute.
2. Compute its closure.
3. Add subsets of remaining attributes in increasing size, computing each closure.
4. Keep a set only if it determines the whole schema and is minimal.
5. Stop exploring any branch already containing a found candidate key.

With A -> B and B -> A on ABC, C is compulsory. Both AC and BC are candidate keys. Finding AC does not finish the question. No dependencies at all makes the whole schema the only candidate key.

Worked example W16. On ABCD, let A→B, B→A, C→D. C never appears on a right side, so every key must contain C. C+={C,D}, still missing A,B. AC+ and BC+ both equal ABCD. Neither A nor C alone is a key, and neither B nor C alone is a key. Candidate keys: AC,BC. Their union makes A,B,C prime; D is nonprime. A minimal key can be longer than another minimal key in a different schema. Enumerate all minimal choices, not only the first one found.

Check before moving on. Did you find every minimal key? Mark all prime attributes only after that search.

R17 Count superkeys

If a schema has n attributes and exactly one candidate key of size k, there are 2^{n-k} superkeys. With multiple keys, count the union of their supersets. Use inclusion-exclusion or enumerate small subsets. Do not add supersets counts without removing overlap.

For AC and BC on ABC: the superkeys are AC, BC and ABC, giving three. Summing two counts of two would count ABC twice.

Worked example W17. With keys AC and BC on ABCD, supersets of AC are AC,ABC,ACD,ABCD. Supersets of BC are BC,ABC,BCD,ABCD. The overlap is ABC,ABCD. Hence 4+4−2=6 superkeys. If an extra E can be obtained from A, the same keys have eight supersets each and four overlaps, giving 12. An extra unconstrained attribute instead becomes compulsory in every key. Counting optional attributes requires knowing which attributes the key already determines.

Check before moving on. Did you remove overlapping supersets? Does an added column become optional or compulsory?

R18 Minimal cover and equivalence

Split right sides into single attributes. Test each proposed extraneous left-side attribute by computing closure under the current dependency set. Remove it only if the dependency remains implied. To test a redundant dependency, remove that dependency and compute the left-side closure using the others. Repeat until no further reduction applies. A minimal cover need not be unique.

Two dependency sets are equivalent if every dependency in each set follows from the other. Test both directions. Comparing their written left sides is insufficient.

Worked example W18. F={A→BC,B→C,AB→D}. Split A→BC into A→B and A→C. In AB→D, B is extraneous because starting from A, A→B already supplies it; simplify to A→D. A→C is redundant because A→B→C still derives C after that rule is removed. A minimal cover is {A→B,B→C,A→D}. To check equivalence, test every original rule from the cover and every cover rule from the original set. Remove A→D from the cover and A+ loses D, so equivalence fails. Removing a rule while testing redundancy is essential.

Check before moving on. For redundancy, remove the tested rule before calculating closure. Check equivalence both ways.

R19 Highest normal form

Find all candidate keys and prime attributes first. Assume 1NF only if values meet the question's atomic-domain assumption.

- 2NF prohibits a nonprime attribute depending on a proper subset of a candidate key.
- 3NF requires, for every nontrivial implied X -> A with a singleton right side, that X be a superkey or A be prime.
- BCNF requires every nontrivial implied determinant to be a superkey.

When F is a cover, testing each dependency of F with singleton right sides suffices to check 3NF and BCNF. For 2NF, test partial dependencies using closures of proper subsets of candidate keys; they may not be written explicitly. Check BCNF first, then 3NF, then 2NF. One violation disproves a normal form. Single-attribute candidate keys automatically avoid partial-key dependencies. “No transitive dependency” is too imprecise to replace the 3NF definition.

Worked example W19. On ABC with AB→C and C→B, the candidate keys are AB and AC. Every attribute is prime. C→B violates BCNF because C+={B,C} omits A. It does not violate 3NF because B is prime. The highest form is 3NF. Contrast A→B,B→C on ABC: A is the sole key, B and C are nonprime, and B→C violates 3NF. It is still 2NF because a single-attribute key has no nonempty proper subset causing a partial dependency. To see a 1NF-only case, take ABC with AB→C and A→C: AB is the key, C is nonprime, and A is a proper part of the key determining C. Atomic cells do not by themselves guarantee 2NF.

Check before moving on. Show a violating determinant and nonprime dependent. Do not use the vague rule no transitive dependencies.

R20 Binary lossless decomposition

For a split into R1 and R2 whose union is R, compute C = R1 intersect R2. The split is lossless under F exactly when C determines all of R1 or all of R2. Compute C+ using F and test containment. The test concerns all permitted instances, not only the sample table.

For R(ABC), A -> B, the split AB/AC is lossless because A determines AB. With only A -> B, splitting AB/BC is not guaranteed lossless: B does not determine AB or BC. A join can invent combinations, called spurious tuples.

Worked example W20. For ABC with A→B, split into AB and AC. The intersection is A and A+ contains AB, so the split is lossless. Split instead into AB and BC: their intersection B determines neither component. Take legal original rows (a1,b1,c1),(a2,b1,c2). AB joined with BC also creates (a1,b1,c2),(a2,b1,c1). Those extra combinations are spurious. The original relation still satisfies A→B. If B→C were additionally required, this counterexample would be illegal and the AB/BC split would be lossless.

Check before moving on. Use the intersection's closure under the full FDs. A single successful sample join does not prove losslessness.

R21 Nonbinary losslessness

Do not apply the binary intersection test once to a three-way split and declare it done. Use a sequence of justified binary decompositions when that sequence yields the stated result; otherwise use the chase.

Make a row per component and a column per original attribute. Use the shared distinguished symbol a_j for an attribute present in a component, and a fresh b_ij otherwise. For each X -> Y, rows agreeing on every X column must agree on Y. Merge symbols, preferring a distinguished symbol when present, and repeat. A row entirely of distinguished symbols establishes losslessness. Termination without such a row means the decomposition is lossy under these FDs. Practise one complete worked tableau before using this recipe.

Worked example W21. For ABCD, F={A→B,B→C}, components AB,BC,AD, make a chase tableau. Row AB is (aA,aB,b13,b14); row BC is (b21,aB,aC,b24); row AD is (aA,b32,b33,aD). A→B makes rows AB and AD agree on B, so b32 becomes aB. Now all three rows agree on B. B→C makes their C symbols aC. Row AD becomes (aA,aB,aC,aD), so the decomposition is lossless. Without B→C, its C remains b33 and no row becomes entirely distinguished. The chase then establishes that losslessness is not guaranteed. A convenient binary derivation gives the same positive result: split ABCD into ABC/AD using A→BC, then ABC into AB/BC using B→C.

Check before moving on. Write the full tableau and symbol changes. One entirely distinguished row is the success criterion.

R22 Dependency preservation

Project F+ onto each component: retain dependencies using only its attributes. Include implied local dependencies, not just original dependencies that happen to fit. Let G be their union. For each original X -> Y, compute X+ under G. The decomposition preserves dependencies exactly when all these tests succeed. Projected dependencies are already implied by F, so the reverse inclusion follows.

Lossless and dependency-preserving are different requirements. Check them separately.

Worked example W22. With A→B,B→C on ABC, the split AB/AC is lossless because A determines AB. Projected rules include A→B in AB and the implied A→C in AC. But under their union, B+={B}; B→C cannot be checked locally. It is not dependency-preserving. Split into AB/BC instead: the local rules are A→B and B→C, so all original rules remain enforceable, and B determines BC, making the split lossless too. Projection means rules from F+, not merely the written rules that fit.

Check before moving on. Include implied local rules. Losslessness and preservation are separate answers.

R23 Construct a decomposition

For BCNF, choose a violating X -> Y inside the current relation and split it into X union Y and the original attributes minus (Y minus X). Recompute projected dependencies and repeat. These splits are lossless; preservation is not guaranteed.

For a standard 3NF synthesis, find a minimal cover, make relations for its determinants and right sides, remove contained relations, and add a relation containing a candidate key if none already does. This construction gives a lossless dependency-preserving 3NF decomposition under functional dependencies. Explain those guarantees separately from whether a particular output also happens to be BCNF.

Worked example W23. Start ABC with AB→C,C→B. The BCNF violation C→B gives components BC and AC. Their common C determines BC, so the split is lossless. The projected local rules cannot derive AB→C, so preservation is lost. For 3NF synthesis, the cover produces ABC and BC; remove contained BC and keep ABC. ABC contains a candidate key, so no extra key relation is needed. It is 3NF and preserves the rules, although it is not BCNF. A decomposition's guarantees must be checked separately: a cleaner component is not proof of preserved constraints.

Check before moving on. For synthesis, check containment removal and whether some component contains a candidate key.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q46 · Infer FDs

Official question, PDF page 29. Published key: **A; B; D**.

Given U→V,U→W,V→X,WX→Y,WX→Z, (VW)+ starts V,W, adds X from V, then Y,Z from WX. Thus VW→YZ and VW→Y hold. WX→YZ follows directly from the two given rules. No rule adds U to (VW)+, so VW→U fails. Correct options: A,B,D.

DA-2025-Q16 · Why preservation matters

Official question, PDF page 11. Published key: **C**.

If a constraint cannot be checked using component relations' local dependencies, information from components may need to be joined to test it. The intended operator is join, C. This is a statement about checking dependencies after decomposition; it does not assert that every decomposed query requires a join.

DA-2025-Q57 · All keys then BCNF

Official question, PDF page 31. Published key: **B; D**.

A+ gets B,C,E,F, then E gets D,G: all attributes. (BC)+ gets A and hence everything. E+={E,D,G} is not a key. Any key without A must get A through BC→A, so BC is the other minimal choice. E→D violates BCNF since E is not a superkey. A is not the only key and E is not a key. Thus B,D.

DA-2026-Q17 · Candidate-key enumeration

Official question, PDF page 12. Published key: **A**.

For A→BC,CD→E,E→A, D never appears on a right side, so every key contains D. AD obtains BC and then E. CD obtains E,A,B. ED obtains A,B,C. Each pair is minimal. BD cannot obtain A,C,E. The complete candidate-key set is AD,CD,ED, matching A.

CS-2024-set1-Q44 · Dependency rules

Official question, PDF page 28. Published key: **B; C; D**.

A wrongly deletes Y from the determinant. Rows (X=1,Y=1,Z=0,W=0) and (X=1,Y=2,Z=1,W=1) satisfy XY→ZW but violate X→ZW. B follows by splitting a right side. For C, XW supplies Y via W→Y, then XY supplies Z. D is transitivity. Correct: B,C,D.

CS-2024-set2-Q56 · Count possible disjoint FD sides

Official question, PDF page 37. Published key: **50 to 50**.

Each of four attributes has three choices: left side, right side, neither. Start with 3^4. Subtract 2^4 choices with empty left and 2^4 with empty right. The all-neither choice was subtracted twice, so add 1. Result: 81−16−16+1=50. These are possible nonempty disjoint syntax choices, not a claim that all 50 hold in one database.

CS-2025-set1-Q47 · BCNF and a lossless split

Official question, PDF page 47. Published key: **B; C**.

Name determines city and owner and is the original candidate key. All given nontrivial determinants are superkeys, so the original is BCNF and therefore also 3NF: A,D are false. Each component has name as a key, so B is true. The intersection name determines both components, so the split is lossless: C is true.

CS-2025-set2-Q46 · Partial and transitive violations

Official question, PDF page 35. Published key: **C; D**.

For R1(ABCDE), A and D cannot be derived from other attributes, so AD is the candidate key. D→E determines a nonprime attribute from a proper part of that key, violating 2NF and hence 3NF. For R2(ABCD), C is the candidate key since C→A→B,D. A→B and A→D have a non-superkey determinant and nonprime dependents, violating 3NF. The statements identifying both as not 3NF are C,D. A single-attribute key avoids 2NF violations, not 3NF violations.

CS-2022-Q14 · A convention-sensitive normal-form claim

Official question, PDF page 12. Published key: **A**.

The official answer is A. Under the usual exam convention excluding nontrivial empty determinants, a binary relation's only possible nontrivial FD is one attribute determining the other; that determinant is a key. However, if ∅→A is allowed on AB, ∅ need not determine B, so a binary relation need not be BCNF. Do not teach A as unconditional. B is false: AB→C,C→B gives all-prime attributes but violates BCNF. C is false because a relation can have all attributes prime. D is false because BCNF splitting need not preserve dependencies, as W23 shows.

CS-2022-Q31 · Count superkeys

Official question, PDF page 22. Published key: **8 to 8**.

With AB→C,BC→D,C→E on ABCDE, neither A nor B occurs on a right side, so both are compulsory. AB+ obtains C,D,E, giving the unique candidate key AB. C,D,E are optional additions to a superkey. There are 2^3=8 superkeys.

Before the exam

Find every key. Check losslessness and preservation separately. Include implied local dependencies. Mark assumptions.

Chapter 5 · ER models: turn a story into constraints

A student belongs to one department. A department can have many students. A student can have several phone numbers and can enrol in several courses. These are three different facts; the diagram must preserve each.

Vocabulary. An entity is a distinguishable object. An entity set collects entities of one type. A relationship associates entities. Cardinality constraints bound how many partners an entity can have. Participation says whether every entity must participate. Total participation means at least one; it does not mean exactly one. A weak entity needs its owner's identifier together with its partial key.

R24 Read a diagram and bound relationship sizes

Translate each edge into a sentence specifying minimum and maximum partners. Check the notation legend; arrow conventions differ. For many-to-one from A to B, each A has at most one B. If A participates totally, each A has exactly one B. If B is also total, there must be enough A entities to cover all B entities. For a plain binary many-to-many relationship, there are at most |A||B| pairs; if both sides are total and there are no additional restrictions, at least max(|A|,|B|) pairs are needed. Check feasibility and zero-sized sets separately.

Worked example W24. There are 3 students and 2 clubs. In a many-to-many membership relationship with total participation on both sides, each student needs a club and each club needs a student. At least 3 pairs are needed and 6 are possible. A minimum instance is (s1,c1),(s2,c1),(s3,c2); the maximum has every student paired with every club. Change it to many-to-one from Student to Club with total Student participation: each student has exactly one club, so there are exactly 3 pairs. If Club is also total, both clubs must be represented among those three choices. One student and two clubs would make both-side total participation impossible in the many-to-one version.

Check before moving on. Write minimum and maximum partners for each side. Check whether both totals can be satisfied before giving a numeric bound.

R25 Map entities and relationships to tables

Make a table per strong entity. Store composite attributes as their components. Usually do not store derived attributes unless specified. Give a multivalued attribute its own table with the owner's key and value. Give a weak entity a table keyed by owner key plus partial key.

For one-to-many, put the one-side key into the many-side table; total many-side participation requires non-nullness. For many-to-many, use a relationship table with both identifiers and relationship attributes. For one-to-one, choose a side for the reference and enforce uniqueness; participation influences the choice. For ternary relationships, preserve the participating tuple and stated key constraints rather than reducing automatically to binary links.

Worked example W25. For Student(ID,Name,DeptID), Department(DeptID,Name), Course(CourseID,Title), give Student.DeptID a reference to Department. If every student belongs to a department, the reference is NOT NULL. Enrolment(StudentID,CourseID,Grade) represents the many-to-many course relationship, with key (StudentID,CourseID). StudentPhone(StudentID,Phone) represents a multivalued attribute, with the pair as key when duplicate phone facts are not meaningful. A weak Dependent with partial key DependentName gets Dependent(StudentID,DependentName,Age), keyed by (StudentID,DependentName). Total

participation in its identifying relationship is necessary because a dependent cannot be identified without its owner. A composite Address can be stored as separate street/city columns; a derived Age is normally computed from a stored birth date. For a ternary Supplier-Part-Project fact, retain a table containing all three IDs. Its key follows stated constraints; three separate pair tables do not generally preserve which supplier supplied which part to which project.

Check before moving on. For each table, identify its key and references. Distinguish one composite value from independently repeated values.

R26 Minimum relations and missing information

Draw the proposed tables, keys and references. Check that every original fact can be represented without inventing partners or duplicating independent multivalued information. Merge tables only when cardinality, participation and normalization permit it. Counting entity boxes alone misses multivalued attributes, weak entities and many-to-many relationships.

Worked example W26. Two independent entities E1 and E2 have a many-to-many link. E2 also has a multivalued phone attribute. The tables are E1, E2, E2Phone and Link(E1ID,E2ID): four. Total participation on both sides requires links but does not make the many-to-many relationship mergeable. If the link becomes many-to-one from E1 to E2 and E1 is total, store E2ID in E1 and remove Link. The minimum becomes three tables. For a one-to-one relationship, a UNIQUE foreign key on an appropriate side represents the upper bound; choose the side using participation and whether unmatched entities must be kept.

Check before moving on. A merge must preserve every fact and avoid duplicate independent information. Total does not mean one-to-one.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2026-Q61 · Minimum tables

Official question, PDF page 39. Published key: **4 to 4**.

Create E1 and E2 tables for their single-valued attributes. E2's multivalued A22 needs its own table containing E2's key A21 and A22. The many-to-many R12 needs a separate table containing both entity keys. Total participation requires each entity to appear in some R12 row but does not change the many-to-many upper bound. With no extra FDs permitting a merge, the minimum is four relations.

CS-2024-set1-Q20 · Entities versus relationships

Official question, PDF page 14. Published key: **D**.

Instructor, Course, Student and Classroom are entity types. Teaches links Instructor-Course; Registration links Student-Course; Allocation links Course-Classroom; Guides links Instructor-Student. Diagram iv has exactly those endpoints and entity/relationship roles, giving D. Diagram i swaps Student and Registration roles; ii substitutes Projects; iii connects Allocation to Registration instead of Classroom. Read each edge back into the original sentence.

CS-2024-set2-Q20 · Weak-entity participation

Official question, PDF page 14. Published key: **A**.

A weak entity requires an owner to complete its identifier. Every weak-entity instance must therefore participate in its identifying relationship: total participation, A. This does not imply that every owner has a weak entity, nor that a weak entity's partial key is globally unique. The owner's key and the partial key together identify it.

Before the exam

Convert the diagram into sentences before counting. For every relation in the mapping, state what one row means. Check key constraints, total participation, multivalued attributes and whether an independent entity can remain unmatched.

Chapter 6 · Files, indexes and B+ trees

Imagine paper records stored in boxes. Opening a box has a cost. An index is a smaller directory telling us which box to open. A tree node can hold many directory entries, so we can reach a record after opening relatively few boxes.

Vocabulary. A block/page is a unit of storage transfer in the stated model. Blocking factor is records per block. A search key is the indexed attribute set; it need not be a candidate key. A dense index has entries covering every indexed search-key occurrence or distinct value as specified by its layout. A sparse index omits some search-key entries and relies on ordering. A clustering index concerns the data's physical grouping. Use the question's terminology and layout where textbooks differ.

R27 Blocking factor and storage size

For fixed-length unspanned records of size r in blocks of size B, after any stated overhead, bfr = floor(B/r). For N records, blocks = ceil(N/bfr). Records cannot straddle blocks in this model. Spanned or variable-length records require their own packing rule. Use actual sizes and distinguish bits from bytes and KB conventions.

Example: 100 records of 100 bytes in 1024-byte blocks need floor(1024/100)=10 records per block and ten blocks. Do not use ceil(10000/1024), which assumes different packing.

Worked example W27. For 101 unspanned records of 100 bytes in 1024-byte blocks, floor(1024/100)=10 records fit per block. Ten blocks hold 100 records; the last record requires an eleventh. Dividing total bytes by block size and rounding gives 10, but incorrectly packs records across boundaries. With a 50-byte header in every block, usable bytes become 974, so only 9 records fit and ceil(101/9)=12 blocks are needed. A heap file has no search-key order; an ordered file can support ordered searching; a hash-organized file places records by a hash value. Those organization choices affect access, not the arithmetic packing rule.

Check before moving on. Use floor for capacity and ceil for the number of containers. Mark usable bytes after overhead.

R28 Single and multilevel index size

Identify whether the lowest-level entries represent records, distinct values or data blocks. Write the bytes in one entry, including keys and pointers. Compute entries per index block with floor. Use ceil to calculate the blocks required. For multilevel indexing, repeat with one entry per block below until the top fits in one block. Track the entry layout at each level and count the requested levels, not data blocks by accident.

Worked example W28. 10,000 records of 100 bytes in 1000-byte blocks need 1000 data blocks. A sparse lowest-level index has one 20-byte anchor entry per data block, so 50 entries fit per index block. Its 1000 entries need 20 blocks. A second level has 20 anchors and fits in one block. There are 2 index levels and, with no cache, a lookup reads 2 index blocks plus 1 data block. If the lowest level is instead dense with one entry per record, it needs 200 blocks, then 4 blocks, then 1 root block: 3 index levels. Entry counts must start from the stated layout.

Check before moving on. Does an entry represent a record, a distinct value or a data block? Count index levels separately from the data fetch.

R29 B+ tree capacities

Draw one internal node with p child pointers and p-1 separators. If pointer bytes are P, key bytes K and available bytes B, require pP+(p-1)K <= B. Solve for the largest integer p. Leaf layout differs: account for each key/record pointer and any leaf-link overhead stated in the question.

With B=4096, P=10 and K=11, 21p-11 <=4096 gives p=195. A record pointer's size belongs in the leaf calculation, not this internal-node equation.

Worked example W29. An internal node with p child pointers has p−1 separator keys. With available bytes 1024, pointer size 4 and key size 8, 4p+8(p−1)≤1024 gives 12p≤1032 and p≤86. Check p=86: it uses 1024 bytes exactly. p=87 uses 1036 and fails. A leaf with 8-byte keys, 12-byte record pointers and a 4-byte next-leaf pointer satisfies 20q+4≤1024, so q=51. Child-pointer and record-pointer sizes play different roles. The earlier provisional answer 129 was incorrect and has been corrected.

Check before moving on. Test your proposed maximum by substitution, then show that adding one entry does not fit.

R30 Tree height and number of nodes

State whether height counts edges or levels. Compute leaf capacity and internal fanout separately. For the minimum possible height, fill nodes to maximum capacity and calculate how many leaves, parents and successive ancestors are needed. For occupancy bounds, apply minimum occupancy separately to nonroot nodes; the root has exceptions. Distinct indexed records are normally represented in B+ leaves; internal separators are not extra records. B-trees and B+ trees use different layouts.

Worked example W30. For 10,000 distinct records, maximum leaf capacity 50 and internal fanout 100, at least 200 leaves are needed. Their parents can be packed into 2 internal nodes, and one root points to those. Minimum height is 3 levels, or 2 edges. With only 5000 records, 100 leaves fit under one root, giving 2 levels. If a 3-level tree has minimum nonroot fanout 50 and minimum leaf occupancy 25, its root needs at least 2 children, giving at least 2×50×25=2500 records. A root that is itself a leaf uses its own exception. Do not apply the half-full rule to every root situation.

Check before moving on. State levels versus edges, leaf capacity, nonroot occupancy and root exceptions before multiplying.

R31 Tree insertion and deletion

Descend using separator comparisons. Insert in sorted leaf order. On overflow, split according to the stated convention and copy the appropriate leaf separator upward. Internal splits promote a separator rather than using the leaf copy rule. Propagate overflow up to the root and create a new root if necessary.

For deletion, remove the leaf entry, repair affected separators, then resolve underflow through the stipulated borrowing/merging rules. Propagate repair upward and shrink a redundant root. If the question leaves a consequential split or merge convention unspecified, identify the ambiguity instead of assuming all variants produce the same root.

Worked example W31. Use minimum-of-right-subtree separators, leaf capacity 2, internal fanout 3 and a leaf split with two entries left and one right. Start with root [7] and leaves [1,4] | [7,10]. Insert 2: split [1,2,4] into [1,2] | [4], copy 4 up, root becomes [4,7]. Insert 3: split [1,2,3] into [1,2] | [3], copy 3 up. The temporary internal keys [3,4,7] overflow. Promote middle key 4 and create root [4], internal children [3] | [7], leaves [1,2] | [3] | [4] | [7,10]. Delete 10: the right leaf [7] still meets minimum occupancy 1. Delete 7: it underflows, its sibling [4] cannot lend, so merge. The right internal node then has only one child; merge it with the left

internal node and collapse the old root. New root [3,4] points to leaves [1,2] | [3] | [4]. Other split conventions can produce other valid layouts; this example states its convention explicitly.

Check before moving on. Copy leaf separators, promote internal separators and repair root height. Follow the source's routing convention.

R32 Index choice and access cost

Hash indexes support equality lookup in the standard model. Ordered/B+ indexes also support ranges. Distinguish an equality join from a range predicate. For a composite ordered index, reason about the leading attributes; an arbitrary trailing-column predicate does not automatically get a direct range search.

Count index pages read on the path, subtract only explicitly cached levels, and add required data pages. A covering index can avoid fetching data pages. For ranges, include the relevant leaf pages and record pages; clustering changes the record-page cost. Heap scan, ordered-file binary search and hashing have different stated assumptions. Avoid one universal disk-access formula.

Worked example W32. A rating>3.4 predicate needs ordered access, so a B+ index supports it while a standard hash index does not provide a range search. GenreName='Comedy' is equality and can use either. Equality joins on identifiers can also use either under the simple exam model. A three-level B+ lookup with the root cached reads 2 remaining index pages and 1 data page: 3 I/Os. If the required columns are covered by the index, no data fetch is needed and the count becomes 2. A composite ordered index (Dept,Salary) directly supports a salary range within one fixed department; a predicate on Salary alone does not automatically provide the same leading-prefix search. Range costs also depend on matching leaf pages and whether matching records are physically clustered.

Check before moving on. Identify equality, range, leading-prefix use, caching and coverage. Count each page only once.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q55 · Equality versus range

Official question, PDF page 37. Published key: **A; B**.

The rating filter is a range and requires the B+ option in the standard model. Genre.Name='Comedy' is equality, so a hash or B+ index can serve it. The identifier links are equality joins. A uses B+ throughout; B uses a hash for Genre.Name while keeping ordered access for rating. C and D use hashing where the rating range is needed, so they fail. These are suitability claims, not guaranteed optimal query plans.

DA-2026-Q32 · Internal-node fanout

Official question, PDF page 21. Published key: **195 to 195**.

A node has p child pointers of 10 bytes and p−1 keys of 11 bytes. Require $10p+11(p-1) \le 4096$, or $21p \le 4107$. The maximum integer p is 195. At 196 pointers it uses 4105 bytes, exceeding 4096. The 12-byte record pointer is a leaf-layout detail, not an internal child-pointer size.

DA-2026-Q41 · Insertion with the printed separator convention

Official question, PDF page 26. Published key: **A**.

The printed root [5,8] sits above leaves [1,5],[7,8],[9,12], so its separators follow the left-child maxima, rather than W31's right-child minima. Insert 3 into the first leaf: it overflows to [1,3,5]. Splitting into [1,3] and [5] makes the temporary parent separators [3,5,8]. The internal node has too many children and splits around 5. The new root stores 5, giving A. Options with 3,5,8 describe a temporary overflowing node, not the repaired root.

CS-2024-set1-Q21 · Root occupancy

Official question, PDF page 15. Published key: **A**.

The ordinary half-full occupancy requirement has a root exception. A root can have only two children while the tree is small, or be a single leaf. Nonroot internal and leaf nodes still follow their stated minima. Only the root exception is correct: A.

CS-2025-set2-Q57 · Leaf split then parent split

Official question, PDF page 43. Published key: **33 to 33**.

The target leaf becomes [20,22,23,25,26]. With the smallest-right-key copy convention, split it into [20,22] and [23,25,26], copying 23 into parent [7,19,33,44]. The parent temporarily becomes [7,19,23,33,44]. Promote 23 in its internal split; the right parent has keys [33,44]. It is the parent of the leaf containing 25, and its smallest key is 33. The question asks for this parent key, not the root's smallest key.

CS-2023-Q62 · Sparse index and exact requested I/O

Official question, PDF page 65. Published key: **6 to 6**.

Unspanned 100-byte records give floor(1024/100)=10 per block, so 25,000 records use 2500 data blocks. The sparse index has 2500 anchors of 15+5=20 bytes. It fits 51 entries per index block and needs

ceil(2500/51)=50 blocks. Under the question's stipulated binary-search formula, ceil(log2 50)=6 accesses identify the target data block. Reading that data block would be an extra access, but is not requested. The downloaded file's PDF page is 65 even though its printed historical footer says 58.

Before the exam

Draw the byte layout or tree state before using a formula. Distinguish separators from leaf records and data fetches from identifying a data block. State the height and routing conventions. Match the index to the predicate before discussing cost.

Chapter 7 · Warehouses, hierarchies and measures

A shop records sales by product, city and date. A manager wants totals by state and month. We are changing the detail of the same measurements. This is the setting for dimensions, hierarchies and aggregation.

Vocabulary. A dimension describes a way to group observations. A measure is the quantity summarized. Grain says what one fact row represents. A hierarchy connects detailed categories to broader categories. A cuboid represents one choice of aggregation level per dimension. The ALL level summarizes a whole dimension.

R33 Schema and grain

Identify the fact table, its grain, dimension references and measures. A star uses comparatively denormalized dimension tables; a snowflake splits dimensions into related tables. Count only what the stated model requires. Joining a fact to a one-to-many dimension relation can duplicate measures, so test multiplicity before summing.

Worked example W33. A fact row Sale(LineID,ProductID,CityID,DayID,Amount) represents one sale line. Product, City and Day are dimensions; Amount is a measure. A star has this fact table plus three flat dimension tables. If Product's category information is moved to a separate Category table referenced by Product, that part becomes snowflaked. Specify grain before aggregating: one sale line is not one daily product total. If product A has a sale of 100 and two tags while B has a sale of 50 and one tag, joining all tags produces 100,100,50. Its sum 250 double-counts A; the true fact sum is 150. A dimension join is safe for summing only when its multiplicity preserves the fact rows.

Check before moving on. Say what one fact row represents. Check join multiplicity before summing a measure.

R34 OLAP operations

Write the grouping before and after the operation. City to State is roll-up; State to City is drill-down. Fixing one dimension value is a slice. Restricting a subcube by several dimension conditions is a dice. Changing the presentation axes is a pivot. Decide from the actual change, not the number of output rows alone.

Worked example W34. A report grouped by State becomes grouped by City: drill-down. City becomes State: roll-up. Keeping only Country='IN' is a slice of the cube. Restricting several product and country values is a dice. Swapping row and column presentation is a pivot; the grouping values and measure can stay unchanged. The operation follows which levels and filters change. More output rows alone do not establish drill-down because a new date range could also add rows.

Check before moving on. Write the before and after grouping levels, filters and presentation. Classify the change from those facts.

R35 CUBE groups and hierarchy counts

Without hierarchies, d dimensions yield 2^d grouping sets. This counts grouping sets, not output rows. If all combinations of n_i values occur, CUBE returns product(n_i+1) groups including totals. For two dimensions with three and two values: 6 detail groups, 3 first-dimension totals, 2 second-dimension totals and 1 grand total, giving 12.

With missing combinations, count distinct groups for each grouping set and add them. For independent linear hierarchies, multiply the available levels in each dimension, including ALL. Branching hierarchies require enumerating the valid level choices under the specified model.

Worked example W35. Facts are (A,IN,10),(B,IN,20),(A,US,30). The Product×Country grouping set has 3 observed groups; Product has 2; Country has 2; the grand total has 1. CUBE returns 8 rows across 4 grouping sets. It does not invent the missing (B,US) detail group. Adding that combination makes the count 4+2+2+1=9. In a separate hierarchy-count question, Geography has City,State,Country,ALL; Product has Item,Category,ALL; Time has Day,Month,Year,ALL. Independent linear level choices give 4×3×4=48 cuboids. Member counts such as 100 cities do not enter this level count. Branching hierarchies need their valid alternatives counted explicitly.

Check before moving on. Are you counting grouping sets, observed rows or hierarchy-level choices? Use the formula for that quantity only.

R36 Measures and correct roll-up

An additive measure can be summed over all relevant dimensions. A semi-additive measure can be summed over some dimensions but not others: an account balance is not generally summed over dates. A non-additive measure needs another aggregation rule.

An average is algebraic: carry sum and count, then divide. An unweighted average of unequal-sized subgroup averages is wrong. Distinct counts and medians generally cannot be recovered from subgroup counts/medians alone. For measure computation questions, identify the quantity and the sufficient summaries before calculating.

Worked example W36. Two sales average 100 and eight sales average 50. Carry sum/count: first group sum 200,count 2; second sum 400,count 8. Combined mean=(200+400)/(2+8)=60. The unweighted average 75 is wrong. SUM,COUNT,MIN,MAX are distributive: summaries can be combined with the same type of aggregation. AVG is algebraic because a fixed-size sum/count summary suffices. Exact MEDIAN is holistic in the general case because a fixed number of subgroup medians cannot determine it. This classification differs from additivity across dimensions: sales amounts are additive; a day's account balances can be summed across accounts but normally not across dates, so balance is semi-additive; a ratio such as margin needs its numerator and denominator rather than summed ratios. Distinct customer counts need overlap information: sets {1,2} and {2,3} have counts 2 and 2 but their union has count 3, not 4.

Check before moving on. Which summaries can actually reconstruct the requested result? Distinguish additive dimensions from distributive/algebraic/holistic computation.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2025-Q46 · CUBE output rows

Official question, PDF page 25. Published key: **D**.

The question supplies all combinations of 3 values in one dimension and 2 in the other. Detail grouping contributes 6 rows. The single-dimension groupings contribute 3 and 2; ALL contributes 1. Total 6+3+2+1=12, giving D. There are four grouping sets, not four output rows. The product (3+1)(2+1) works here because all combinations occur.

DA-2026-Q18 · A hierarchy operation

Official question, PDF page 12. Published key: **D**.

The report changes from Country totals to State detail. State is below Country in the hierarchy, so this is drill-down: D. Roll-up would go the other way. Slice/dice restrict values, and pivot changes presentation; neither describes a change to a finer hierarchy level by itself.

DA-2026-Q43 · Count hierarchy cuboids

Official question, PDF page 28. Published key: **A**.

The diagram gives three levels each for Time, Geography and Item. Include ALL as a fourth level in each dimension. Choosing one level independently in each gives 4×4×4=64 cuboids, written 4^3 in option A. Counting only the drawn three levels omits grand aggregation; 2^3 would count binary include/exclude choices rather than the supplied hierarchies.

Before the exam

Start with grain and the requested aggregate. For CUBE, label exactly what is being counted. For roll-up, carry enough information to reconstruct the measure and check for repeated fact rows. Learn both measure classifications; they answer different questions.

Chapter 8 · Data types and preprocessing

Two students have marks 40 and 80. If we measure achievement on a different scale, the underlying ordering can remain the same. But assigning labels such as Low and High loses numerical detail. These are different transformations, and the exam may ask exactly what each preserves.

Vocabulary. Nominal categories have no inherent order. Ordinal categories have an order but not necessarily equal gaps. Interval scales support meaningful differences; their zero is arbitrary. Ratio scales also have a meaningful zero. Discrete/continuous describes a different distinction. Numeric category codes do not automatically permit meaningful averaging.

R37 Numeric normalization

Identify the specified rule and statistics. Min-max mapping to [a,b] is a+(x-min)(b-a)/(max-min). Z-score is (x-mean)/standard deviation. Decimal scaling divides by 10^j under the specified size convention. Substitute with units consistently and round only at the end. Constant-valued data requires an explicit handling rule for a zero denominator. Population and sample standard deviations are different if the question requires computing one.

Worked example W37. A department code labels a category, so it is nominal even if stored as an integer. A satisfaction rating Low/Medium/High is ordinal: it has order but unspecified gaps. Celsius temperature is interval: differences are meaningful but zero is not absence of temperature. Mass in kg is ratio: its zero and ratios have physical meaning. Discrete versus continuous is a separate distinction. For numeric x=60, minimum 20, maximum 80, mean 50 and standard deviation 10, min-max to [0,1] gives (60−20)/(80−20)=2/3. Z-score gives (60−50)/10=1, meaning one standard deviation above the mean. To [-1,1], the result is −1+2×(2/3)=1/3. Decimal scaling, with the smallest j making max|x|/10^j<1 and maximum absolute value 80, uses j=2 and gives 0.60. Constant data make min-max's denominator zero; the question must specify a handling convention. Numeric normalization changes a scale; relational normalization changes a schema under dependencies.

Check before moving on. Name the scale, transformation and statistics. Do not divide by zero or average arbitrary category codes.

R38 Discretization and bin smoothing

Sort the data. Equal-width bins divide the numeric range evenly; equal-frequency bins aim for similar counts. State how boundary values and ties are assigned. A question may supply that convention. For smoothing, replace values by the bin mean, median or nearest boundary as specified; preserve repeated observations in those calculations. Discretization replaces values with bin labels. Smoothing and labelling are different operations.

Worked example W38. Sort 0,1,2,3,8,9. Equal-width bins [0,3),[3,6),[6,9] contain {0,1,2},{3},{8,9}. Their means are 1,3,8.5, so smoothing gives 1,1,1,3,8.5,8.5. Equal-frequency bins with two observations each are {0,1},{2,3},{8,9}; their means are 0.5,2.5,8.5. Smoothing gives 0.5,0.5,2.5,2.5,8.5,8.5. Replacing observations with labels Bin1/Bin2/Bin3 is discretization, not mean smoothing. Boundary and tie rules must be explicit: a value 3 goes into the second stated equal-width interval. For boundary smoothing using a bin's observed minimum and maximum, replace each member with its nearer boundary and follow the stated tie rule.

Check before moving on. State intervals and ties, preserve repeated observations, and distinguish numeric smoothing from bin labels.

R39 Sampling

Identify with/without replacement and whether selection is simple, stratified, systematic or cluster-based. Determine the sampling unit before counting probabilities. For strata, use their actual sizes and sampling allocations; equal numbers from unequal strata do not make an unweighted sample mean representative. Do not import general Markov-chain sampling machinery merely because the DA preprocessing syllabus includes sampling.

Worked example W39. A population has 90 members in stratum A and 10 in B. We sample five from each stratum and observe sample means 10 and 30. The population-mean estimate is $0.9\times10+0.1\times30=12$. The raw sample mean is $(5\times10+5\times30)/10=20$; equal sample allocation overrepresents B unless weights are applied. A simple random sample chooses individual units uniformly. A stratified design samples within each stratum. A cluster design might choose villages and include every household in the chosen villages; the initial sampling unit is a village. A systematic design takes every kth unit after a random start; periodic ordering can make a realized sample unrepresentative. For two ordered draws from five IDs, sampling without replacement gives $5\times4=20$ outcomes; with replacement gives $5\times5=25$. Do not confuse an ordered sampling sequence with an unordered subset.

Check before moving on. Identify the sampling unit, replacement rule, stratum sizes and weights before calculating.

R40 Compression

Distinguish exact reconstruction from an approximation: lossless versus lossy. For a stated encoding, count actual encoded bits, including overhead where required. Define the ratio direction before computing it. Repeated-run encoding saves space on long runs but can expand short runs. Reduced precision or representative bins usually discard information. No compression scheme shortens every possible input while remaining lossless.

Worked example W40. An original 1000-byte file is encoded as a 600-byte payload plus a 100-byte header. Encoded size is 700 bytes. Define compression ratio as original/encoded: $1000/700=10/7$. Saving is $(1000-700)/1000=30\%$. For run-length encoding of AAAABBCD using 8 bits for a symbol and 8 bits for its count, there are four runs, costing $4\times16=64$ bits. The original eight characters also cost 64 bits; an 8-bit header makes the encoding 72 bits, so it expands. Lossless encoding must permit exact reconstruction; replacing a bin's values by their mean usually does not. No lossless scheme shortens every possible input: four two-bit strings cannot all be mapped distinctly into the three shorter strings empty,0,1. Overhead and the data's pattern both matter.

Check before moving on. Include all overhead. State the ratio's direction and whether exact reconstruction is possible.

Previous-year questions, worked

Read the original question and options in the linked official paper. These are original explanations, not replacement question statements. PDF page numbers count from the first page of the downloaded file.

DA-2024-Q27 · Z-score with consistent units

Official question, PDF page 17. Published key: **B**.

Use the mean 96,000 and standard deviation 21,000, not the minimum and maximum. For 106,000, $z=(106000-96000)/21000=10/21\approx0.47619$, matching B. The result is dimensionless because the numerator and denominator use the same units. Round at the end.

Before the exam

Match the numerical operation to what it preserves or loses. Use units consistently. Keep sampling weights and compression overhead visible. These are data transformations, not database schema-normal-form problems.

Sources and coverage boundaries

The official 2027 syllabus and the downloaded official papers/keys establish scope and historical answers. The accompanying source ledger records question IDs, PDF pages, links and caveats. DA papers are 2024,2025,2026. CS selections are 2022,2023,2024 Sets 1/2 and 2025 Sets 1/2.

Database System Concepts author resources provide reference definitions for relations, SQL, ER design, dependencies and indexing. Han, Kamber and Pei author resources provide the reference framework for warehouse measures and preprocessing. All teaching examples, new exercises and explanations here are independently written; no textbook slides are reproduced.

This first-pass exam pack teaches FD-based 1NF through BCNF and their decomposition methods. It does not add a transactions, locking, recovery or backend-engineering course. MVD/4NF/5NF are outside this stated first-pass scope; do not read this as proof that a broad future syllabus can never require them. Historical CS/IT acquisition leads remain in the source archive and are not silently treated as solved questions.

Terminology check

Often confused	Distinction
Schema / instance	permitted structure and rules / current rows
Degree / cardinality	attributes / tuples
Minimal / minimum	no removable member / smallest size
Search key / candidate key	indexed columns / minimal unique identifier
X+ / F+	attributes obtained from X / all implied dependencies
Lossless / preserving	reconstruct rows / enforce FDs locally
Grouping sets / output rows	aggregation choices / groups actually produced
Relation normalization / numeric normalization	schema dependencies / scale transformation